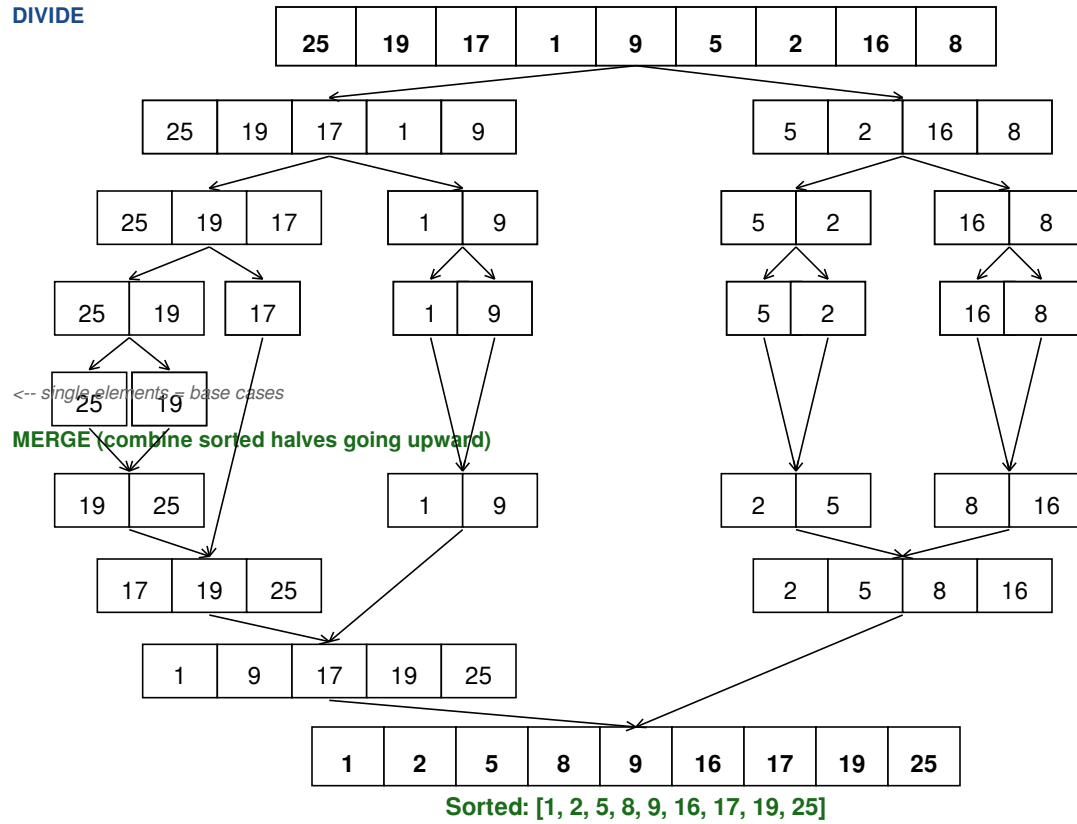


(20 pts) 1. MergeSort recursion tree on the numbers below.

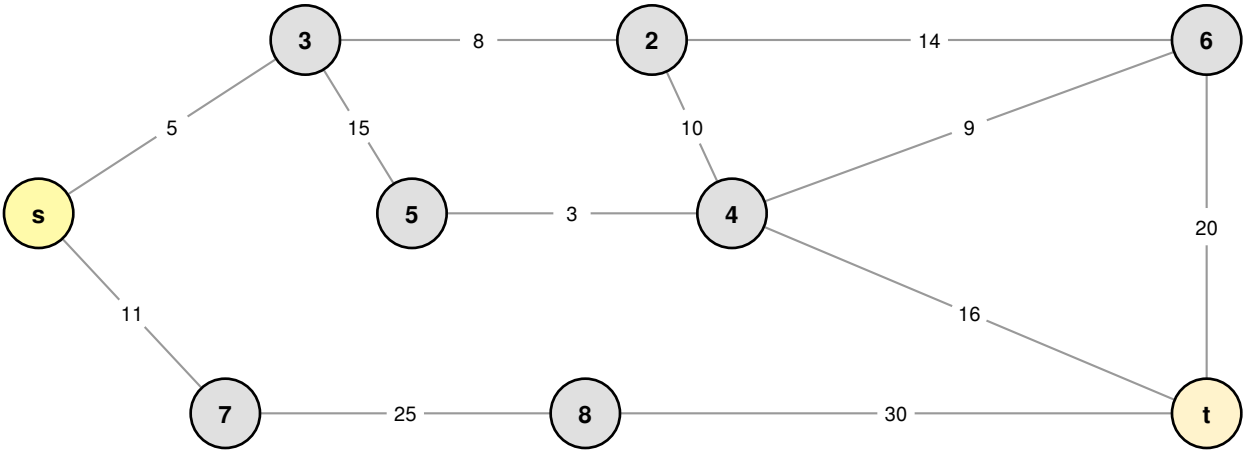
Array: [25, 19, 17, 1, 9, 5, 2, 16, 8]

DIVIDE



(60 pts) 2. Minimum Spanning Tree — Prim's, Kruskal's, Reverse Delete

Graph edges: s-3(5), s-7(11), 3-2(8), 3-5(15), 2-6(14), 2-4(10), 5-4(3), 4-6(9), 4-t(16), 6-t(20), 7-8(25), 7-8(25), 8-t(30)

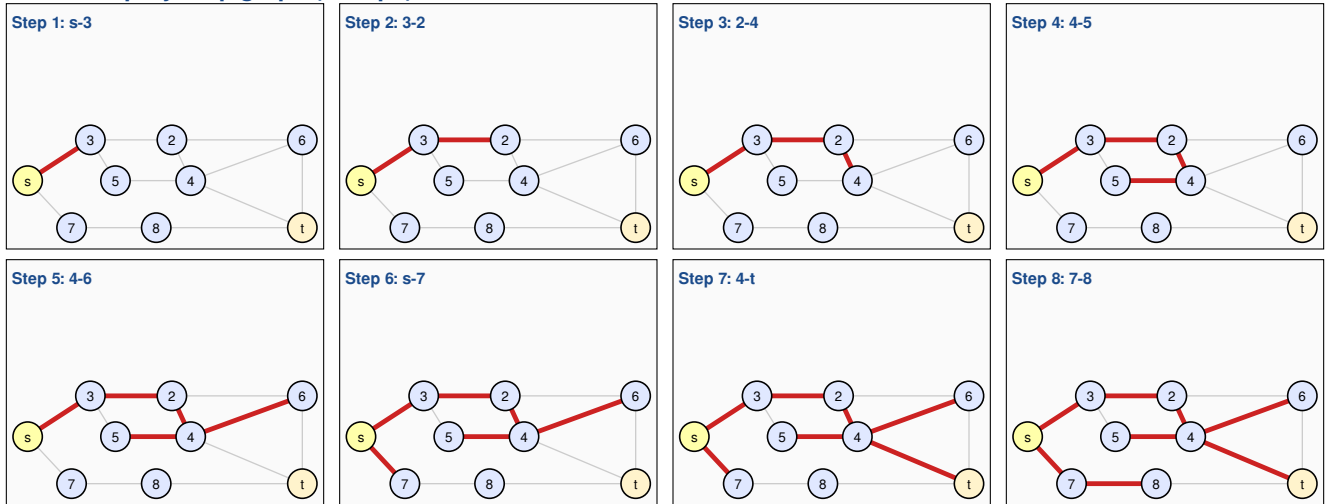


PRIM'S ALGORITHM (start from vertex s)

Step	Edge Added	Wt	Visited Set (MST so far)	Reason chosen
Start	—	—	{ s }	Begin at s
1	s - 3	5	{ s, 3 }	Min edge from s: s-3(5) < s-7(11)
2	3 - 2	8	{ s, 3, 2 }	Min crossing: 3-2(8)
3	2 - 4	10	{ s, 3, 2, 4 }	Min crossing: 2-4(10) < others
4	4 - 5	3	{ s, 3, 2, 4, 5 }	Min crossing: 4-5(3) smallest!
5	4 - 6	9	{ s, 3, 2, 4, 5, 6 }	Min crossing: 4-6(9) < 2-6(14)
6	s - 7	11	{ s, 3, 2, 4, 5, 6, 7 }	Min crossing: s-7(11)
7	4 - t	16	{ s, 3, 2, 4, 5, 6, 7, t }	Min crossing: 4-t(16) < 6-t(20)
8	7 - 8	25	{ s, 3, 2, 4, 5, 6, 7, t, 8 }	Last vertex — all connected!

Prim's MST Edges: s-3, 3-2, 2-4, 4-5, 4-6, s-7, 4-t, 7-8 | Total Weight = 5+8+10+3+9+11+16+25 = 87

Prim's step-by-step graph (8 steps):



KRUSKAL'S ALGORITHM

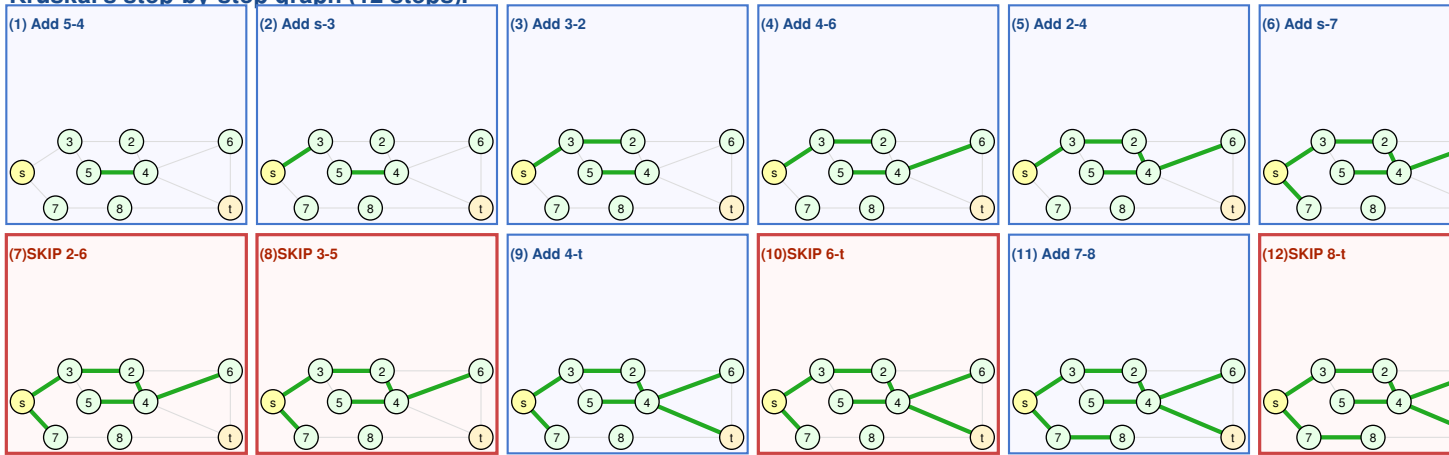
Sort ALL edges ascending by weight. Add each edge if it does NOT form a cycle. Use union-find to detect cycles.

Sorted: 5-4(3) s-3(5) 3-2(8) 4-6(9) 2-4(10) s-7(11) 2-6(14) 3-5(15) 4-t(16) 6-t(20) 7-8(25) 8-t(30)

Step	Edge	Wt	Cycle?	Action	Components / Status
1	5-4	3	No	ADD	{s}{3}{2}{6}{4,5}{7}{8}{t}
2	s-3	5	No	ADD	{s,3}{2}{6}{4,5}{7}{8}{t}
3	3-2	8	No	ADD	{s,3,2}{6}{4,5}{7}{8}{t}
4	4-6	9	No	ADD	{s,3,2}{4,5,6}{7}{8}{t}
5	2-4	10	No	ADD	{s,3,2,4,5,6}{7}{8}{t}
6	s-7	11	No	ADD	{s,3,2,4,5,6,7}{8}{t}
7	2-6	14	YES	SKIP	2 and 6 already in same component
8	3-5	15	YES	SKIP	3 and 5 already in same component
9	4-t	16	No	ADD	{s,3,2,4,5,6,7,t}{8}
10	6-t	20	YES	SKIP	6 and t already connected
11	7-8	25	No	ADD	{s,3,2,4,5,6,7,t,8} — DONE
12	8-t	30	YES	SKIP	All 9 vertices connected

Kruskal's MST Edges: 5-4, s-3, 3-2, 4-6, 2-4, s-7, 4-t, 7-8 | Total Weight = 3+5+8+9+10+11+16+25 = 87

Kruskal's step-by-step graph (12 steps):

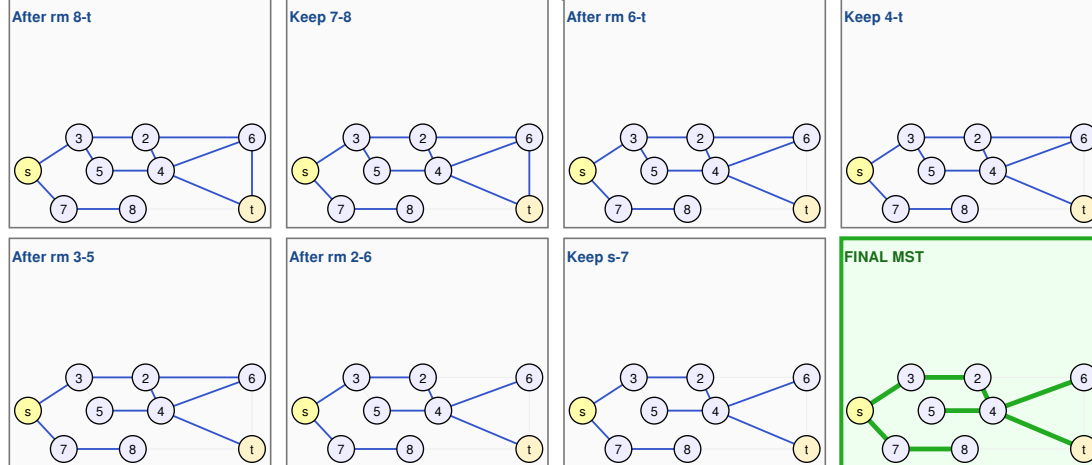


REVERSE DELETE ALGORITHM

Start with ALL edges. Consider heaviest first. Remove an edge ONLY if the graph remains connected.

Step	Edge	Wt	Decision	Reason / Justification
1	8-t	30	REMOVE ?	Graph stays connected — 8 still reachable via 7-8 and s-7
2	7-8	25	KEEP ?	Would disconnect vertex 8 (8-t already removed, no path left to 8)
3	6-t	20	REMOVE ?	t connected via 4-t; 6 connected via 4-6. Still connected.
4	4-t	16	KEEP ?	Would disconnect t entirely (8-t already removed)
5	3-5	15	REMOVE ?	5 reachable via 4-5 then 2-4 then 3-2 then s-3
6	2-6	14	REMOVE ?	6 still reachable via 4-6
7	s-7	11	KEEP ?	Would disconnect vertices 7 and 8 from the rest
8	2-4	10	KEEP ?	Would split graph: {s,3,2} disconnected from {4,5,6,7,t,8}
9	4-6	9	KEEP ?	6 would be disconnected (2-6 already removed)
10	3-2	8	KEEP ?	Would disconnect vertex 2 from s and 3
11	s-3	5	KEEP ?	Would disconnect s from the whole graph
12	5-4	3	KEEP ?	Would disconnect vertex 5

Reverse Delete MST: s-3, 3-2, 2-4, 4-5, 4-6, s-7, 4-t, 7-8 | Total Weight = 87

Reverse Delete — step-by-step graph (remaining edges at each step):

All 3 Algorithms produce the SAME MST — Total Weight = 87

MST Edges: s-3(5), 3-2(8), 2-4(10), 4-5(3), 4-6(9), s-7(11), 4-t(16), 7-8(25)

(20 pts) 3. Quicksort on S = {6, 5, 3, 2, 7, 1, 4}

Using PARTITION(A,p,r): pivot = A[r] (last element). i starts at p-1. j scans p to r-1.
 If A[j] ≤ pivot: i++, swap A[i] and A[j]. After loop: swap A[i+1] and A[r], return i+1.

PASS 1: PARTITION([6, 5, 3, 2, 7, 1, 4]) pivot x = A[r] = 4, i = p-1 = -1

p/j	1	2	3	4	5	6	
	6	5	3	2	7	1	4
							Initial: i=-1, j=0
	j					r	
	6	5	3	2	7	1	4
							j=0: 6 > 4 → no swap; j=1
		j				r	
	6	5	3	2	7	1	4
							j=1: 5 > 4 → no swap; j=2
i		j				r	
6	5	3	2	7	1	4	
							j=2: 3 ≤ 4 → i=0, swap A[0] ↔ A[2]
	i		j			r	
3	5	6	2	7	1	4	
							j=3: 2 ≤ 4 → i=1, swap A[1] ↔ A[3]
	i		j			r	
3	2	6	5	7	1	4	
							j=4: 7 > 4 → no swap
	i		j		r		
3	2	6	5	7	1	4	
							j=5: 1 ≤ 4 → i=2, swap A[2] ↔ A[5]
	i	$i+1$				r	
3	2	1	5	7	6	4	
							End of loop: swap A[i+1=3] ↔ A[r=6]
			$pivot$				
3	2	1	4	7	6	5	
							4 is now at index 3 — FINAL POSITION!

Recurse LEFT — PARTITION([3, 2, 1]), pivot = 1, i = -1

p/j	r		
3	2	1	
			Initial: pivot=1, i=-1, j=0
		r	
3	2	1	
			j=0: 3 > 1 no swap; j=1: 2 > 1 no swap
	$i+1$	r	
1	2	3	
			End: swap A[0] ↔ A[2] → 1 in final position
	$pivot$		
[2, 3]: pivot=3. j=0: 2 ≤ 3 → i=0, swap A[0] ↔ A[0]. End: place 3. → [2, 3] done.			

Recurse RIGHT — PARTITION([7, 6, 5]), pivot = 5, i = -1

p/j	r		
7	6	5	
			Initial: pivot=5, i=-1, j=0
		r	
7	6	5	
			j=0: 7 > 5 no; j=1: 6 > 5 no
	$i+1$	r	
5	6	7	
			End: swap A[0] ↔ A[2] → 5 in final position
	$pivot$		
[6, 7]: pivot=7. j=0: 6 ≤ 7 → i=0, swap A[0] ↔ A[0]. End: place 7. → [6, 7] done.			

Final sorted array: